

# EC-110 Computing

## Fundamentals Experiment # 7

Introduction to Programing.

### 1. Objective/s:

- What is C++ Language
- Compiler
- Structure of C Program
- Console Program
- Creating a new Project in Visual Studio
- Lab Task

### 2. What is C Language

C is a high-level and general-purpose programming language that is ideal for developing firmware or portable applications. Originally intended for writing system software, C was developed at Bell Labs by Dennis Ritchie for the Unix Operating System in the early 1970s.

Ranked among the most widely used languages, C has a compiler for most computer systems and has influenced many popular languages – notably C++.

### 3. Compiler

Computers understand only one language and that language consists of sets of instructions made of ones and zeros. This computer language is called *machine language*.

A single instruction to a computer could look like this:

|  |      |
|--|------|
|  | 0    |
|  | 1110 |

A particular computer's machine language program that allows a user to input two numbers, adds the two numbers together, and displays the total could include these machine code instructions:

|  |      |
|--|------|
|  | 0    |
|  | 1110 |
|  | 1    |
|  | 0100 |
|  | 0    |
|  | 1110 |
|  | 1    |

|   |      |
|---|------|
|   | 0100 |
| 0 | 1111 |
| 1 | 0000 |

As you can imagine, programming a computer directly in machine language using only ones and zeros is very tedious and error prone. To make programming easier, high level languages have

Department of CE HITEC University, Taxila

BSCE – 1<sup>st</sup> Semester Lab – 07 Fall 2021

been developed. High level programs also make it easier for programmers to inspect and understand each other's programs easier.

This is a portion of code written in C++ that accomplishes the exact same purpose:

1

```
int a, b, sum;

cin >> a;
cin >> b;

sum = a + b;
cout << sum << endl;
```

2

3

4

5

6

7

Even if you cannot really understand the code above, you should be able to appreciate how much easier it will be to program in the C++ language as opposed to machine language.

Because a computer can only understand machine language and humans wish to write in high level languages high level languages have to be re-written (translated) into machine language at some point. Special programs called compilers, interpreters, or assemblers that are built into the various programming applications do this.

C++ is designed to be a compiled language, meaning that it is generally translated into machine language that can be understood directly by the system, making the generated program highly efficient. For that, a set of tools are needed, known as the development toolchain, whose core are a compiler and its linker.

## 4. Structure of a Program

The best way to learn a programming language is by writing programs. Typically, the first program beginners write is a program called "Hello World", which simply prints "Hello World" to your computer screen. Although it is very simple, it contains all the fundamental components C++ programs have:

|   |                                                                                                                                                |              |
|---|------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 1 | <pre>// my first program in C++<br/>#include &lt;iostream&gt;<br/><br/>int main()<br/>{<br/>    std::cout &lt;&lt; "Hello<br/>World!"; }</pre> | Hello World! |
| 2 |                                                                                                                                                |              |
| 3 |                                                                                                                                                |              |
| 4 |                                                                                                                                                |              |
| 5 |                                                                                                                                                |              |
| 6 |                                                                                                                                                |              |
| 7 |                                                                                                                                                |              |

The left panel above shows the C++ code for this program. The right panel shows the result when the program is executed by a computer. The grey numbers to the left of the panels are line numbers to make discussing programs and researching errors easier. They are not part of the program.

Let's examine this program line by line:

Line 1: `// my first program in C++`

Department of CE HITEC University, Taxila

BSCE – 1<sup>st</sup> Semester Lab – 07 Fall 2021

Two slash signs indicate that the rest of the line is a comment inserted by the programmer but which has no effect on the behavior of the program. Programmers use them to include short explanations or observations concerning the code or program. In this case, it is a brief introductory description of the program.

Line 2: `#include <iostream>`

Lines beginning with a hash sign (#) are directives read and interpreted by what is known as the *preprocessor*. They are special lines interpreted before the compilation of the program itself begins. In this case, the directive `#include <iostream>`, instructs the preprocessor to include a section of standard C++ code, known as *header iostream*, that allows to perform standard input and output operations, such as writing the output of this program (Hello World) to the screen.

Line 3: A blank line.

Blank lines have no effect on a program. They simply improve readability of the code.

Line 4: `int main ()`

This line initiates the declaration of a function. Essentially, a function is a group of code statements which are given a name: in this case, this gives the name "main" to the group of code statements that follow. Functions will be discussed in detail in a later chapter, but essentially, their definition is introduced with a succession of a type (`int`), a name (`main`) and a pair of parentheses (`()`), optionally including parameters.

The function named `main` is a special function in all C++ programs; it is the function called when the program is run. The execution of all C++ programs begins with the `main` function, regardless of where the function is actually located within the code.

Lines 5 and 7: `{` and `}`

The open brace (`{`) at line 5 indicates the beginning of `main`'s function definition, and the closing brace (`}`) at line 7, indicates its end. Everything between these braces is the function's body that defines what happens when `main` is called. All functions use braces to indicate the beginning and end of their definitions.

Line 6: `printf("Hello World!");`

This line is a C++ statement. A statement is an expression that can actually produce some effect. It is the meat of a program, specifying its actual behavior. Statements are executed in the same order that they appear within a function's body.

This statement has two parts: First, `printf`, which is a function used to print something on screen. Second, a sentence within quotes ("Hello world!"), is the content inserted into the standard output.

Notice that the statement ends with a semicolon (`;`). This character marks the end of the statement, just as the period ends a sentence in English. All C++ statements must end with a semicolon character. One of the most common syntax errors in C++ is forgetting to end a statement with a semicolon.

You may have noticed that not all the lines of this program perform actions when the code is

Department of CE HITEC University, Taxila

BSCE – 1<sup>st</sup> Semester Lab – 07 Fall 2021

executed. There is a line containing a comment (beginning with `//`). There is a line with a directive for the preprocessor (beginning with `#`). There is a line that defines a function (in this case, the `main` function). And, finally, a line with a statements ending with a semicolon (the insertion into `cout`), which was within the block delimited by the braces (`{ }`) of the `main` function.

The program has been structured in different lines and properly indented, in order to make it easier to understand for the humans reading it. But C++ does not have strict rules on indentation

or on how to split instructions in different lines. For example, instead of

1

```
int main ()
{
    printf(" Hello
World!"); }
```

2

3

4

We could have written:

```
int main () { printf(" Hello
World!"); }
```

all in a single line, and this would have had exactly the same meaning as the preceding code.

In C++, the separation between statements is specified with an ending semicolon (;), with the separation into different lines not mattering at all for this purpose. Many statements can be written in a single line, or each statement can be in its own line. The division of code in different lines serves only to make it more legible and schematic for the humans that may read it, but has no effect on the actual behavior of the program.

Now, let's add an additional statement to our first program:

1

```
// my second program in C++
#include <iostream>

int main ()
{
    printf(" Hello World!");
    printf("I'm a C++ program
"); }
```

```
Hello World! I'm a C++ program
```

2

3

4

5

6

7

8

Preprocessor directives (those that begin by #) are out of this general rule since they are not statements. They are lines read and processed by the preprocessor before proper compilation begins. Preprocessor directives must be specified in their own line and, because they are not statements, do not have to end with a semicolon (;).

## 5. Console Program

Console programs are programs that use text to communicate with the user and the environment, such as printing text to the screen or reading input from a keyboard.

Department of CE HITEC University, Taxila

BSCE – 1<sup>st</sup> Semester Lab – 07 Fall 2021

Console programs are easy to interact with, and generally have a predictable behavior that is identical across all platforms. They are also simple to implement and thus are very useful to learn the basics of a programming language: The examples in these tutorials are all console programs.

The way to compile console programs depends on the particular tool you are using

The easiest way for beginners to compile C++ programs is by using an Integrated Development Environment (IDE). An IDE generally integrates several development tools, including a text editor and tools to compile programs directly from it.

Here you have instructions on how to compile and run console programs using different free Integrated Development Interfaces (IDEs):

| IDE                          | Platform             | Console programs                                            |
|------------------------------|----------------------|-------------------------------------------------------------|
| <b>Code::blocks</b>          | Windows/Linux/Mac OS | <a href="#">Compile console programs using Code::blocks</a> |
| <b>Visual Studio Express</b> | Windows              | Compile console programs using VS Express 2013              |
|                              |                      |                                                             |
| <b>Dev-C++</b>               | Windows              | <a href="#">Compile console programs using Dev-C++</a>      |

## 6. Creating a new Project in Visual Studio

**To create a project and add a source file**

1. Create a project by pointing to **New** on the **File** menu, and then clicking **Project**. 2. In the

**Visual C++** project type's pane, click **Win32**, and then click **Win32 Console Application**.

3. Type a name for the project. By default, the solution that contains the project has the same name as the project, but you can type a different name. You can also type a different location for the project. Click **OK** to create the project.

4. In the **Win32 Application Wizard**, click **Next**, select **Empty Project**, and then click **Finish**.

5. If **Solution Explorer** is not displayed, on the **View** menu, click **Solution Explorer**. 6. Add a new source file to the project, as follows.

a. In **Solution Explorer**, right-click the **Source Files** folder, point to **Add**, and then click **New Item**.

b. In the **Code** node, click **C++ File (.cpp)**, type a name for the file, and then click **Add**.

The .cpp file appears in the Source Files folder in **Solution Explorer**, and the file is opened in the Visual Studio editor.

Department of CE HITEC University, Taxila

BSCE – 1<sup>st</sup> Semester Lab – 07 Fall 2021

7. In the file in the editor, type a valid C++ program that uses the Standard C++ Library, or copy one of the sample programs and paste it in the file.

8. Save the file.

9. On the **Build** menu, click **Build Solution**. The **Output** window displays information about the compilation progress, for example, the location of the build log and a message that indicates the build status.

10. On the **Debug** menu, click **Start without Debugging**. If you used the sample program, a command window is displayed and shows whether certain integers are found in the set.

## 7. Tasks

1. Write and run a program to print your first name on the first line, your middle name on the second line and your last name on the third line using only “printf” statement.
2. Write and run a program that performs the following steps:
  - Assigning value to the radius  $r$ .
  - Calculating the area:  $A=(\pi)r^2$



- Displaying the Area.
- Calculating the circumference using the formula:  $C = 2(\pi)r$ .
- Displaying the circumference.